ТЗ: веб-интерфейс

Что делаем

Одна страница. Менеджер пишет товар одной строкой, жмёт кнопку и видит: категорию, как сужался выбор кодов КТРУ, найденные характеристики, итоговый код или уточняющий вопрос, чего не хватает. Стиль UI-кита Портала поставщиков: за интерфейс 10 баллов из 100.

- UI-кит Портала поставщиков в Figma: элементы и иконки, забирать оттуда, а не рисовать самим.
- Сайт портала: цвета и разметку страницы сохраняем как там, это прямо в критериях.

Фич сверх этого не делаем, баллов за них нет.

Критерии, которые зависят от фронта

Формулировки из официальных критериев (docs/briefing.md, «Критерии оценки целиком»).

Целиком на фронте, блок «Интерфейс», 10 баллов:

Критерий	Что оценивают	Баллы
Интуитивность, цвета и разметка портала	Как быстро пользователь понимает сервис и сам проходит путь до кода КТРУ, с сохранением цветовой схемы и разметки страницы портала	4
Отображение и редактирование	Понятно показаны извлечённые характеристики, значения, единицы, и их можно редактировать	2
Наглядность сужения кодов	Пользователь понимает, как система идёт от описания к коду	2
Уведомления и подсказки	Сообщения помогают понять разъяснение или ошибку	2

Считает бэк, но жюри увидит это только через страницу:

Критерий	Что оценивают	Баллы	Что от фронта
Работоспособный прототип	Реально работающий веб-сервис, весь сценарий: от текста до унифицированных характеристик и конечного кода КТРУ	10	весь путь проходится на странице, без консоли
Поэтапное сужение категорий КТРУ	Логично и правильно сокращает число возможных кодов	6	шаги сужения видны
Неоднозначность и нехватка данных	Правильное поведение, когда пользователь дал мало информации	3	вопрос с вариантами, список недостающих обязательных

Критерий	Что оценивают	Баллы	Что от фронта
Несоответствия и несуществующие значения	Проверка характеристик не только на опечатки, но и на соответствие	8	замечание с обоснованием видно рядом с характеристикой
Качество автоисправления	Правильно исправляет и приводит описание к виду, пригодному для подбора	3	видно «было → стало»
Скорость	Насколько быстро обрабатывается запрос при приемлемом качестве	5	страница не тормозит ответ, есть индикатор загрузки

Что пользователь должен уметь по слайдам 4 и 5 постановки:

- ввести описание на обычном языке, с опечатками и неполными данными, и получить характеристики, значения, единицы и код КТРУ;
- видеть, как система поэтапно сужает коды, и получать уведомление, если код однозначно не определить, с предложением дополнить текст и конкретными шагами;
- получать уведомление об ошибках в характеристиках с обоснованием, почему они некорректны и какие нужно добавить;
- видеть, какие характеристики унифицированы, какие значения и единицы сопоставлены, и следить за подбором кода в реальном времени.

Что ещё сказали ведущие:

- за систему сядет обычный менеджер, а не программист: всё понятно без объяснений;
- у каждой правки обоснование, «чтобы человек не угадывал, почему система что-то выкинула, что-то поменяла»;
- у каждой характеристики видна метка источника: КТРУ, портал или новая;
- нехватку обязательных характеристик подсвечивать;
- подозрительное в тексте пользователя подсвечивать.

Из чего состоит

- **Сервер.** Один файл на Python, вызывает `run(text)` из pipeline/run.py и отдаёт результат как JSON. Стек на выбор фронта, что-то простое (FastAPI, Flask). Запуск через uv.
- **Страница.** HTML, CSS, JS. Сборщики и фреймворки не обязательны.

Пайплайн уже работает на настоящем справочнике, отвечает за 0,1 с, заглушка не нужна. Чтобы он запустился, нужен data/processed/ktru.json (как собрать, в README).

Контракт API

POST /api/run

Запрос:

```
{"text": "молоток слесарный 500 г"}
```

Сервер проверяет ввод сам, внутри пайплайна проверок нет:

- text пустой или из одних пробелов → 400 `{"error": "Опишите товар"}`;
- text длиннее 1000 символов → 400 `{"error": "Слишком длинное описание"}`;
- пайплайн упал → 500 `{"error": "Не удалось разобрать описание"}`.

Ответ 200, тип Result из contracts/pipeline.py:

Поле	Тип	Что это
category	"clothing", "furniture", "tools" или null	категория; null, если не поняли
steps	список Step	шаги сужения по порядку
chars	список FoundChar	найденные характеристики
code	строка или null	итоговый код КТРУ; null, если однозначного нет
question	Question или null	что спросить у пользователя, есть только когда code null и кандидатов несколько
missing_required	список строк	обязательные характеристики, которых нет в тексте
remarks	список Remark	исправления: было, стало, почему. Пока всегда пустой

Step:

Поле	Тип	Что это
what	строка	что сделали, уже по-человечески: «Похоже на Молоток», «Тип: Слесарный»
left	число	сколько кодов осталось после шага

FoundChar:

Поле	Тип	Что это
name	строка	название характеристики
value	строка или число	значение, уже в единице КТРУ
unit	`{"code", "name"}` или null	единица, показывать name
source	"ktru", "portal" или "new"	откуда характеристика
fragment	строка	кусок исходного текста, из которого взято значение

Question: `char` это название характеристики, `options` это варианты ответа строками.

Remark: `was`, `now`, `why`, все строки.

Подписи для показа:

- category: clothing «Одежда», furniture «Мебель», tools «Инструменты»;
- source: ktru «КТРУ», portal «Портал», new «Новая».

Три исхода и как их показывать

1. Код найден (code не null):


```
{
  "category": "tools",
  "steps": [
    {"what": "Похоже на Молоток", "left": 11},
    {"what": "Тип: Слесарный", "left": 1}
  ],
  "chars": [
    {"name": "Тип", "value": "Слесарный", "unit": null, "source": "ktru", "fragment":
"слесарный"},
    {"name": "Масса молотка", "value": 0.5, "unit": {"code": "166", "name":
"Килограмм"}, "source": "ktru", "fragment": "500 г"}
  ],
  "code": "25.73.30.141-00000012",
  "question": null,
  "missing_required": [],
  "remarks": []
}
```


Крупно код, под ним таблица характеристик.

2. Нужно уточнение (code null, question не null):


```
{
  "category": "furniture",
  "steps": [{"what": "Похоже на Стол письменный", "left": 2}],
  "chars": [
    {"name": "Вид материала столешницы", "value": "ЛДСП", "unit": null, "source":
"ktru", "fragment": "ЛДСП"}
  ],
  "code": null,
  "question": {"char": "Тип каркаса", "options": ["Деревянный", "Металлический"]},
  "missing_required": ["Конфигурация стола", "Тип каркаса", "Назначение стола
письменного"],
  "remarks": []
}
```


Вопрос «Уточните: Тип каркаса» и кнопки с вариантами. По нажатию дописать к тексту `Тип каркаса: Деревянный` и отправить запрос заново. Исключение: если char равен «Код КТРУ», в options лежат сами коды, выбранный вариант и есть ответ, перезапрос не нужен.

3. Не нашли (code null, question null):

```
{
  "category": null,
  "steps": [{"what": "Не нашли в КТРУ позицию с похожим названием", "left": 0}],
  "chars": [],
  "code": null,
  "question": null,
  "missing_required": [],
  "remarks": []
}
```


Предупреждение с текстом последнего шага и подсказкой описать товар подробнее.

Что на странице

1. **Ввод.** Поле на всю ширину, кнопка «Подобрать», пара примеров-подсказок под полем, клик по примеру подставляет текст. Пока идёт запрос, кнопка неактивна.
2. **Категория.** Плашка с подписью категории.
3. **Сужение.** Шаги по порядку, у каждого what и число оставшихся кодов. Шаги приходят все сразу, показывать их по одному с небольшой задержкой, чтобы было видно, как сужается выбор. Шаг, после которого остался 0, выделить как ошибку.
4. **Характеристики.** Таблица: название, значение, единица, источник (цветная метка), фрагмент текста. Подсветить фрагменты в исходном тексте.
5. **Итог.** Код, или вопрос с кнопками, или «не нашли» (см. три исхода).
6. **Не хватает.** Если missing_required не пустой, предупреждение со списком.
7. **Исправления.** Если remarks не пустой, список «было → стало, почему».

Пустой ввод ловим на странице до запроса и на сервере.

После встречи с жюри (суббота 15:00)

- Редактирование характеристик в таблице: правка значения и единицы, удаление строки. Контракт для этого обсудим отдельно, сейчас не делаем.
- В remarks появятся исправления опечаток и несоответствий, блок 7 начнёт заполняться.

Правила

- Контракт Result меняем только объявив всем, правка в contracts/pipeline.py. Если для страницы чего-то не хватает, говори Пайплайну, поля результата определяет тот, кто ими пользуется.
- Перед слиянием: `uv run ruff check . && uv run ruff format --check . && uv run ty check`, `uv run pytest`.
- data/ в git не попадает.